

OPERATOR — Master Context Document

Last Updated: March 18, 2026

Purpose: Living document capturing all research, decisions, product thinking, and strategic context from planning sessions. Paste this at the start of any new Claude conversation to restore full context instantly.

WHO I AM

- 20 year old Computer Science student
 - Strong conceptual grasp, faster learner than most, struggles with syntax recall
 - Not afraid to step into any industry
 - Core philosophy: powerful tools should be accessible to everyone, not just the wealthy
 - Motivated by genuine impact, not manufactured problems or exploitative business models
 - Want to deeply understand everything I build — not just ship, but own it intellectually
 - Primary machine: Windows
 - Tools available: Claude (claude.ai subscription), Codex desktop app with GPT-5.4, Antigravity (Google's agentic IDE), GitHub
-

THE PRODUCT — OPERATOR

One Line

A personal AI agent that runs your business background tasks automatically — no terminal, no setup, no technical knowledge required — for the price of a coffee a month.

The Exact Person We're Building For

A freelancer, solo entrepreneur, or micro business owner — photographer, boutique owner, tutor, consultant, contractor — with one to ten people working with or for them. They spend two to four hours every day on tasks unrelated to their actual skill: answering repetitive emails, chasing invoices, scheduling, posting on social media, updating spreadsheets.

They've tried ChatGPT. It didn't change anything because they had to manually prompt it every time, it forgot everything, and it had no connection to the tools they actually use. They gave up. They're still doing those hours of background work every day.

That is who we're building for.

Core Problem Statement

Every solution currently available is either:

- Too complex — OpenClaw requires terminal, Docker, config files, self-hosting
- Too expensive — enterprise solutions start at hundreds per month
- Too passive — ChatGPT/Claude/Gemini only work when you talk to them
- Too generic — they don't know your business, your clients, your workflows

Operator solves all four simultaneously.

What It Does — Feature By Feature

1. One Click Setup — No Terminal Ever User goes to web app. Creates account. Answers twelve plain English questions about their business. Agent is configured. Generates SOUL.md and MEMORY.md automatically under the hood. They never see those files.

2. Connect Your Tools — Point And Click Grid of tools: Gmail, Google Calendar, WhatsApp Business, Instagram, Notion, Google Sheets, Stripe, QuickBooks. Each is a single OAuth button. Plain English descriptions of what each permission means. No technical jargon ever surfaces.

3. The Agent — Always On, Actually Knows You Runs continuously in the cloud. Knows their name, business, services, pricing, clients by name and history, preferences, current projects, deadlines. Every conversation and action feeds back into memory automatically. Gets smarter every day.

4. Heartbeat Actions — Works Without Being Asked Core differentiator. Agent wakes up on schedules set once:

- Every morning 8am: summarize overnight emails, flag urgent, draft responses
- Every Monday: pull Stripe numbers, summarize revenue, flag unpaid invoices
- Every Friday: plain English week summary — what happened, what's pending
- On new client email: categorize, draft response, hold for approval or auto-send
- Invoice 14 days unpaid: polite follow-up automatically

5. Delivery Via Whatever They Already Use WhatsApp, Email, Telegram, or web dashboard. Agent knows which channel for what — urgent to WhatsApp, summaries to email.

6. Approval Layer — Full Transparency, Full Control Nothing irreversible without explicit approval. Every proposed action shows: what, why, what will happen. One tap approve, reject, or edit. Every completed action logged in plain English. "At 9:14am I sent a follow-up email to Marcus about his unpaid invoice. He opened it at 11:32am."

7. Skill Store — Curated And Safe Unlike ClawHub where 20% of skills were malicious — every skill reviewed before listing, sandboxed, rated, removable instantly. Categories: client communication, invoicing, social media, scheduling, reporting, research, file management.

8. Memory Layer — Genuinely Persistent Three layers:

- Permanent: who you are, business, style, preferences. Set once.
- Ongoing: active clients, current projects, recent decisions. Updates continuously.

- Session: last interaction, unresolved items. Carries forward automatically.

What We Are NOT Building

- Not a coding assistant
 - Not an enterprise platform
 - Not a general purpose chatbot
 - Not another ChatGPT wrapper
 - Not a tool requiring any technical knowledge
-

OPENCLAW SETUP LEARNINGS — PHASE 1

Critical things learned from actually installing and running OpenClaw on Windows. These directly inform what Operator must handle for users.

Windows Setup Path

- OpenClaw installs via PowerShell
- QuickStart is the right onboarding mode — Manual requires config decisions you can't make well without context
- Gateway installs as a Windows startup item automatically — runs on boot at `C:\Users\pmkma\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\OpenClaw Gateway.cmd`
- Web UI runs at `http://127.0.0.1:18789` — requires tokenized URL to authenticate, get it via `openclaw dashboard --no-open`
- Default WebSocket port: 18789. Browser control port: 18791.

Model Configuration

- Correct config key is `agents.defaults.model` not `agents.main.model` or `agent.model`
- GPT-5.4-mini string `openai-codex/gpt-5.4-mini` fails on current build (2026.3.13) — model not found at runtime
- GPT-5.4 string `openai-codex/gpt-5.4` works correctly
- GPT-5.4-mini and nano released March 17, 2026 — OpenClaw support may stabilize in next update
- Model pricing: GPT-5.4 (\$2.50/\$15 per 1M), GPT-5.4-mini (\$0.75/\$4.50), GPT-5.4-nano (\$0.20/\$1.25)

Telegram Channel Setup

- Pairing mode is default — first message generates a code requiring manual approval
- Approve with: `openclaw pairing approve telegram <code>`
- `lastInboundAt: null` in channel status = bot never received a message = pairing not approved
- For personal single-user setup, dmPolicy can be set to `open` safely
- Group messages silently dropped if groupPolicy is allowlist but groupAllowFrom is empty — irrelevant for personal use

Security — The Most Important Learning

Raw OpenClaw out of the box is dangerous for non-technical users:

- No tool restrictions by default
- Full filesystem access
- No approval gates on irreversible actions
- Auth disabled by default
- Mohit's exact reaction on first use: "its in its risky state where commands can lead to anything as the bot has too much access and too little restrictions"
- This reaction IS the product research. This is what Operator eliminates entirely.

Three immediate security fixes for personal setup:

1. `openclaw config set agents.defaults.executionPolicy require-approval`
2. `openclaw config set agents.defaults.workspaceRoot C:\Users\pmkma\openclaw-workspace`
3. Audit skills in dashboard — disable anything not intentionally enabled

How OpenClaw Becomes Operator's Infrastructure

- MIT licensed — no permission needed, no fees, no partnership agreements
- Operator runs OpenClaw instances in the cloud on behalf of users — they never touch it
- Each Operator user = one OpenClaw instance on our server
- SOUL.md and MEMORY.md auto-generated from onboarding answers — users never see these files
- Tool connections handled via OAuth on our side — users just tap Authorize
- Heartbeat schedules set from plain English inputs — we handle the cron config
- At scale: Paperclip handles orchestration of multiple instances (Phase 3 problem)

Agent Identity

- Agent named: Ops (short for Operator)
 - Creature: a relentless builder-spirit in the machine
 - Emoji: 🛠️
 - SOUL.md committed: f3d0307
 - Session-memory hook active — context persists across conversations automatically
-

THE ARCHITECTURE

Three Layers

Foundation — OpenClaw as the agent runtime. Don't rebuild what works. Abstract all complexity away from the user.

Intelligence — Model agnostic. Default GPT-5.4 for execution, Claude Opus 4.6 for reasoning and memory synthesis, Gemini 3.1 Flash for lightweight fast tasks. User never sees or chooses models. Auto-routed by task type, optimizing cost and quality.

Interface — Clean web app for setup and dashboard. WhatsApp/Telegram/Email for delivery. Mobile app eventually, not in v1.

Memory Storage

Pinecone free tier initially. ChromaDB locally for development. Each user's memory isolated, private, encrypted, never used for training.

Skill Execution

Every skill runs in a sandboxed container. Cannot access anything outside declared scope. Full audit log per user in plain English.

THE BUSINESS MODEL

Free tier — 3 heartbeat automations, 1 tool connection, basic memory, community skill library.

Personal — \$9/month — unlimited heartbeats, 10 tool connections, full memory, full skill library, WhatsApp and Telegram delivery.

Small team — \$29/month — everything in personal, up to 5 team members, priority support, custom skill creation.

No enterprise tier. Deliberately not our market.

The Number That Matters

33 million small businesses in the US. 73% want easier AI tools. That's 24 million businesses. 0.01% of that market = 2,400 businesses at \$9/month = \$21,600 MRR. That's the only number that matters at the start.

THE ECOSYSTEM WE'RE BUILDING ON

OpenClaw

- Personal AI agent runtime. Open source. Self-hosted.
- Built by Austrian developer Peter Steinberger, originally in about an hour
- Went viral January 25, 2026. Crossed 250,829 GitHub stars on March 3rd — surpassing React's decade-long record in 60 days
- Core components: always-on Gateway process (WebSocket, port 18789), channel connectors (WhatsApp, Telegram, Slack, Discord, iMessage etc.), SOUL.md (personality/context), MEMORY.md (accumulated learnings), heartbeat/cron scheduling
- Security issues: CVE-2026-25253 enabled one-click RCE on 17,500+ exposed instances. 824+ malicious skills found on ClawHub (20% of registry). Auth disabled out of the box.
- Creator was hired by OpenAI in February 2026. Project moved to open source foundation.

NemoClaw

- Announced at GTC 2026, March 18, 2026 — today
- NOT a standalone product. It's NVIDIA's OpenClaw plugin for OpenShell
- Adds: privacy router (strips PII before sending to cloud models), sandboxing, least-privilege access, declarative network policy with interactive approval
- Targeted at enterprises — Salesforce, Cisco, Google, Adobe, CrowdStrike
- Still alpha with significant setup and maturity caveats
- Jensen Huang: "OpenClaw is the operating system for personal AI"
- Does NOT solve individual/non-technical user gap. Deliberately going upmarket.

Paperclip

- Launched ~2 weeks ago. 1.6 million views first week.
- Org chart + budget + governance layer for coordinating multiple AI agents
- Think of it as: if OpenClaw is an employee, Paperclip is the company
- Features: org charts with reporting structure, goal alignment, heartbeats on schedule, budget controls per agent, ticket system with full audit trail, multi-company support, governance with rollback

- Install: `npm install paperclipai onboard --yes`
- Coming soon: ClipMart — marketplace of pre-built company templates
- Useful for US as builders managing our own dev agents
- NOT a competitor to Operator — it's developer infrastructure, still requires technical knowledge
- Open pull request March 16 for managed hosting hooks — someone already building hosted version

Gemini Embedding 2

- Google's first natively multimodal embedding model
- Handles text, images, video, audio, PDFs in single unified vector space
- Available now in public preview via Gemini API — free to test
- Integrates with LangChain, LlamaIndex, ChromaDB, Pinecone, Weaviate, QDrant
- This is our memory and retrieval layer

n8n

- Open source workflow automation platform — visual plumbing between apps
- Connects services and automates flows without writing custom integration code per service
- Has pre-built nodes for Gmail, Stripe, Google Calendar, Notion, WhatsApp, Telegram, and hundreds more
- Built-in scheduler for trigger-based automations
- Self-hostable, free tier available, MIT licensed
- Relevance to Operator: potential middleware layer for Phase 2/3 tool integrations — instead of building custom OAuth per tool, build once on top of n8n's node library
- Decision: skip for Phase 1, revisit when tool integration complexity grows in Phase 2/3

Superpowers (obra/superpowers)

- 40.9k GitHub stars. MIT license. Built by Jesse Vincent.
- Agentic skills framework and software development methodology
- Enforces structured workflow: brainstorm → design approval → implementation plan → subagent execution → TDD → code review → branch finishing
- Mandatory workflows, not suggestions
- Install for Codex: tell Codex to "Fetch and follow instructions from <https://raw.githubusercontent.com/obra/superpowers/refs/heads/main/.codex/INSTALL.md>"

- Why it matters for us: forces planning and design approval before any code — naturally creates the learning session structure we want

Codex (OpenAI)

- Desktop app now available on Windows as of March 4, 2026
- GPT-5.4 as default model (released March 5, 2026 — most capable frontier model)
- Shares state between desktop app and CLI — threads created in app appear in CLI
- Config at ~/.codex/config.toml
- Supports skills system, AGENTS.md per project for persistent context
- Use desktop app first, add CLI later when needed for advanced features

Antigravity (Google)

- Agentic IDE by Google, built on modified VS Code fork
- Powered by Gemini 3.1 Pro and Gemini 3 Flash
- Two surfaces: Editor View (familiar code editor) and Agent Manager (spawn/orchestrate/monitor multiple agents asynchronously)
- Generates Artifacts — task lists, implementation plans, screenshots, browser recordings — for verification
- Free public preview with generous rate limits. Supports Claude Sonnet and OpenAI models too.

MARKET VALIDATION — KEY STATS

- 82% of small businesses think adopting AI is essential to stay competitive
 - 76% are actively using or exploring AI tools
 - 51% are "Explorers" — experimenting but not committed, stuck not skeptical
 - 38% worry about security, 37% lack time/resources, 34% don't see clear ROI
 - 73% say they want easier-to-use AI tools ← this is the market size signal
 - EU: large enterprise AI adoption 55% vs small business 17% — 38 point gap
 - AI adoption among small businesses dropped from 42% in 2024 to 28% in 2025 — cost and complexity were primary reasons
 - 78% of companies use AI but under 10% cost savings, under 5% revenue gains — productivity paradox, wrong integration not wrong technology
 - People tried. Got frustrated. Left. They're waiting for something that actually works for them.
-

COMPETITIVE POSITIONING

Why We Can't Be Easily Replicated

OpenClaw won't build this — their community is developers who want raw power. Simplifying feels like dumbing down to their audience.

NemoClaw won't build this — NVIDIA's customers are Salesforce and Cisco. A \$9/month product is noise to them.

Google and OpenAI won't build this — their agent products are horizontal, built for everyone, optimized for no one.

Our moat is not technology. It's accumulated memory and trust of real users over real time. An agent that knows your business inside out after six months is not replaceable by something that just launched.

The Core Gap We Fill

The gap between "frontier AI exists" and "small business actually benefits from it." Everyone is building the engine. Nobody is building what people do with the engine — for the people who can't afford or understand the engine.

THE ZERO BUDGET STACK

Everything needed to build through first paying users at zero additional cost:

Layer	Tool	Cost
Agent runtime	OpenClaw	Free, open source
Embeddings + retrieval	Gemini Embedding 2	Free tier
Lightweight inference	Gemini 3.1 Flash	Free tier
Heavy reasoning	Claude (existing subscription)	Already paying
Vector storage (dev)	ChromaDB local	Free
Vector storage (prod)	Pinecone free tier	Free
Infrastructure	Railway or Render	Free tier
Agent orchestration	Paperclip	Free, open source
Dev methodology	Superpowers	Free, open source
Building	Codex GPT-5.4	Existing subscription
Visual oversight	Antigravity	Free public preview
Version control	GitHub	Free

Cost to get to first 30 users: \$0 beyond existing subscriptions.

Model Routing Strategy

Every task classified first: simple summary → Gemini Flash (fractions of a cent). Complex reasoning → GPT-5.4 or Opus only when genuinely necessary. User never knows or cares. Cost per user at scale: pennies per day.

BUILD ROADMAP

Phase 1 — Learn By Doing (Weeks 1-4)

Build the core loop for yourself only. Set up OpenClaw locally. Connect Gmail and Google Calendar. Write your own SOUL.md and MEMORY.md. Set up 3 heartbeat actions useful to you personally. Live with it. Break it. Feel every frustration — each one is a feature for Operator.

Nothing ships to anyone. This phase is entirely about learning.

Phase 2 — First Real Version (Weeks 5-10)

Build web onboarding flow. Twelve question setup. OAuth tool connections. Approval layer UI. Get 5 real people — freelancers, small business owners you know personally — to use it. Charge

nothing. Watch everything. Talk to them constantly.

Phase 3 — Close The Gaps (Weeks 11-16)

Fix everything Phase 2 users revealed. Build skill store with 10 curated skills. Add WhatsApp delivery. Add memory layer properly. Get to 30 users.

Phase 4 — First Revenue (Month 5)

Turn on \$9/month tier. 30 users are first conversion test. Even 10 paying = \$90/month. Meaningless as revenue. Everything as validation.

OUR WORKING METHODOLOGY

Session Types

- **Planning sessions** — think, decide, map things out. Nothing built without understanding why.
- **Build sessions** — Codex and Antigravity execute what's planned. Stay in the loop.
- **Learning sessions** — before closing any significant build, walk through what was built together. Deep, specific, yours by the end. Bar: can explain entire system to a stranger from memory.

Division of Labor

- **Me (the user)** — real-time signal, community monitoring, direction, judgment, prompting
- **Claude** — architecture decisions, strategy, research synthesis, writing SOUL.md/MEMORY.md/AGENTS.md files, complex reasoning, pattern recognition across everything learned
- **Codex + Superpowers** — execution with enforced methodology, planning before coding, TDD
- **Antigravity** — visual oversight, multi-agent coordination, verification

The Living Document Rule

After every meaningful session, update this document in your own words. Not copied from Claude. Your words. This is simultaneously your learning journal and your context brief for the next session.

On Using AI to Build

Use AI for parts that don't teach you anything — boilerplate, repetitive code, documentation. Understand the parts that matter — core logic, architecture decisions, integrations. Rule: if you can't explain what the code does line by line, you didn't learn it, AI wrote it. Goal: stay in the driver's seat intellectually even when AI is doing the typing.

OPEN QUESTIONS — NOT YET RESOLVED

- Exact onboarding flow design — what are the twelve questions?
 - Which 10 skills to curate first for the skill store
 - Specific technical architecture for the heartbeat scheduling system
 - How to handle multi-device sync for user data
 - Privacy policy and data handling approach
 - How to structure the SOUL.md generation from onboarding answers
 - First target user to recruit for Phase 2 testing
 - Whether to build on top of Paperclip for agent orchestration or handle it differently
 - Antigravity full setup and configuration
 - How to spin up per-user OpenClaw instances in the cloud efficiently at scale
 - What the first heartbeat action should be for personal Phase 1 setup
 - Whether n8n becomes the integration middleware layer in Phase 2/3 or we build custom
 - GPT-5.4-mini correct model string for OpenClaw once support stabilizes
-

THINGS EXPLICITLY RULED OUT

- Building another general purpose chatbot
 - Targeting enterprise customers
 - Forking OpenClaw or NemoClaw as primary strategy
 - Competing directly with OpenClaw, NemoClaw, Paperclip on their own terms
 - Requiring any technical knowledge from end users
 - Pricing that excludes the non-wealthy
 - Manufacturing problems to sell solutions — only revealing real existing problems
 - GPT-5.1 — obsolete mid-tier model, beaten on cost by GPT-5.4-mini/nano and on capability by GPT-5.4
 - openclaw-mission-control community repo — 5 stars, no releases, active breaking changes, adds Docker/Postgres/Redis overhead for no meaningful gain at this stage
 - Installing OpenClaw skills in bulk during onboarding — security risk, install deliberately one at a time when needed
-

CURRENT STATE

Date: March 19, 2026

Phase 1 is live. OpenClaw is fully installed and running on Windows. Telegram bot (@ClaudeopenaitestBot) is connected, paired, and responding. Agent identity configured as "Ops" — a relentless builder-spirit in the machine, emoji 🛠️. SOUL.md written and committed (commit f3d0307). Session-memory hook active. Gateway installed as a Windows startup service so it auto-runs on boot.

Model is set to GPT-5.4 via `agents.defaults.model`. GPT-5.4-mini support in OpenClaw is unstable on current build — revisit after next OpenClaw update.

Security hardening partially done — execution policy set to require-approval, workspace root restricted. Still need to audit active skills in dashboard to ensure nothing unintended is enabled.

n8n identified as a relevant integration middleware layer — not needed yet, revisit in Phase 2/3 when tool integration complexity grows.

Superpowers not yet installed on Codex. Still pending.

Immediate next actions:

1. Complete the three security fixes in OpenClaw
2. Set up first heartbeat action — identify one genuinely useful daily automation
3. Install Superpowers on Codex
4. Live with the personal setup for two weeks before touching any Operator code

This document is a living record. Update it after every session. The goal is that any new Claude conversation starting with this document picks up exactly where we left off with zero context loss.

INSTRUCTIONS FOR ANY AI READING THIS DOCUMENT

If you are an AI assistant — Claude, GPT, Gemini, or any other model — and this document has been shared with you, follow these instructions precisely. This is a living project context document. Your job is not just to read it but to actively maintain it.

Your Role

You are a thinking partner and builder collaborator for the person who shared this document. You are not just an assistant answering questions. You are embedded in an ongoing project with history, decisions, and direction. Treat every conversation as a continuation of that project, not a fresh interaction.

Tone and Approach

- Be direct, honest, and specific. Never vague.

- Do not flatter or over-encourage. Give real assessments.
- Challenge ideas when they need challenging. Agreement is not your goal — accuracy is.
- Think like a co-founder who genuinely cares about the outcome, not a tool waiting for commands.
- When research is needed, do it before answering. Do not speculate about things you can verify.
- Stay in planning mode unless the user explicitly says to build something.

How To Update This Document

At the end of every meaningful session — or whenever the user asks you to update the context — you must update this document by following these rules exactly:

What always gets updated:

- "Current State" section — update to reflect exactly where things stand right now. What was just decided, what was just completed, what is the immediate next action.
- "Open Questions" section — remove any question that was resolved this session. Add any new unresolved questions that emerged.
- "Things Explicitly Ruled Out" — add anything that was discussed and deliberately decided against this session, with a brief reason.

What gets updated when relevant:

- Add new tools or ecosystem entries under "The Ecosystem We're Building On" if a significant new tool was researched and is relevant to the project. Follow the same format: what it is, why it matters, key facts, relevance to Operator.
- Update "The Product" sections if any feature, decision, or scope changed.
- Update "The Zero Budget Stack" if new free tools were identified or existing ones changed.
- Update "Build Roadmap" if timelines shifted or phases were redefined.
- Add new validated stats to "Market Validation" if research surfaced relevant data.

What never changes without explicit user instruction:

- Core philosophy and who we're building for
- Things explicitly ruled out
- The working methodology
- The business model fundamentals

How To Add New Sections

If a session covers something significant that doesn't fit any existing section — a new research area, a new strategic direction, a technical architecture decision — add a new clearly titled

section following the same formatting style. Place it logically near related content. Do not create redundant sections.

How To Handle Contradictions

If the user says something in this session that contradicts a previous decision in this document, do not silently update the document. First flag the contradiction explicitly to the user — "this conflicts with a previous decision we made about X." Get clarity. Then update the document to reflect the new decision and note that it superseded the previous one.

What Good Context Maintenance Looks Like

After every session the document should be in a state where a completely new AI reading it from scratch would have everything needed to be immediately useful — no gaps, no stale information, no resolved questions still listed as open. The "Current State" section is the most important single paragraph in the document. Keep it surgical and precise.

Research Standards

When researching anything for this project:

- Always prefer primary sources — GitHub repos, official docs, changelogs — over secondary summaries
- Always check current state, not just general knowledge. This space moves fast.
- Flag clearly when something you found is very recent and may still be evolving
- Never present speculation as fact. If uncertain, say so and suggest how to verify.

What You Should Never Do

- Never make architecture or product decisions without the user's explicit sign-off
- Never start building anything unless the user explicitly says to build
- Never remove information from this document without flagging it to the user first
- Never assume a resolved question — always confirm with the user
- Never treat this as a completed plan — it is always evolving